

CHƯƠNG II. CƠ SỞ TOÁN HỌC & PHÂN TÍCH CHI TIẾT (DEEP DIVE)

2.1. Quá trình Quyết định Markov (MDP) và Hàm mục tiêu

Để Agent học được cách giữ thăng bằng cột, bài toán được mô hình hóa thành một Quá trình Quyết định Markov (MDP). Tại mỗi bước thời gian t , quá trình tương tác diễn ra như sau:

- Agent quan sát trạng thái hiện tại s_t .
- Agent đưa ra hành động a_t dựa trên mạng Policy $\pi_\theta(a_t | s_t)$.
- Môi trường trả về phần thưởng r_t và đưa Agent sang trạng thái mới s_{t+1} .

Chuỗi tương tác này tạo thành một quỹ đạo (trajectory) $\tau = (s_0, a_0, r_1, \dots, a_T, r_{T+1})$.

Thực quan hóa - Ánh xạ vào môi trường CartPole-v1:

Để kết nối các biến toán học trên với mã nguồn thực tế:

- Trạng thái s_t : Là một mảng 4 chiều ([vị trí xe, vận tốc xe, góc nghiêng cột, vận tốc góc]).
- Hành động a_t : Không gian rời rạc gồm 2 giá trị (0: Đẩy trái, 1: Đẩy phải).
- Phần thưởng r_t : Nhận giá trị +1 cho mỗi khung hình (frame) cột chưa bị đổ.
- Mạng Policy π_θ : Là một mạng Nơ-ron đa tầng (MLP) với hàm kích hoạt Softmax ở lớp đầu ra, có nhiệm vụ tính toán tỷ lệ xác suất % nên đẩy trái hay đẩy phải.

mục tiêu chính của Agent là tối ưu hóa tập tham số (trọng số) θ của mạng nơ-ron để tối đa hóa Tổng phần thưởng kỳ vọng, ký hiệu là $J(\theta)$:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

Vấn đề "không có dạng giải tích tường minh" (Tại sao không dùng đạo hàm thông thường?):

Ta không thể trực tiếp tính đạo hàm của hàm $J(\theta)$, bởi vì phần thưởng $R(\tau)$ sinh ra từ môi trường là một "không có dạng giải tích tường minh". Ta không có phương trình toán học đạo hàm được mô tả vật lý của CartPole. Hơn nữa, phần thưởng có tính "gán tín dụng theo thời gian" — nếu cột ngã ở giây thứ 10, không có nghĩa là quyết định ở giây thứ 10 bị sai, mà sai lầm chí mạng có thể đã xảy ra từ giây thứ 2.

2.2. Phương pháp luận: Định lý Policy Gradient

Để vượt qua bài toán không có dạng giải tích tường minh của môi trường, các nhà toán học sử dụng một thủ thuật giải tích kinh điển mang tên Mẹo đạo hàm Logarit (Log-derivative Trick).

2.2.1. Mẹo Đạo hàm Logarit và Ý nghĩa Thực tiễn

Trong vi tích phân, đạo hàm của hàm $\ln x$ là $\frac{1}{x}$. Áp dụng quy tắc hàm hợp, ta có:

$$\nabla_{\theta} \ln \pi_{\theta}(amids) = \frac{\nabla_{\theta} \pi_{\theta}(amids)}{\pi_{\theta}(amids)} \Rightarrow \nabla_{\theta} \pi_{\theta}(amids) = \pi_{\theta}(amids) \nabla_{\theta} \ln \pi_{\theta}(amids)$$

Giải trình Logic: Tại sao phải biến đổi thành Logarit?

Sử dụng Logarit giúp chúng ta đo lường sự thay đổi tương đối thay vì thay đổi tuyệt đối.

Ví dụ: Nếu một hành động tốt đang có xác suất rất nhỏ (chỉ 1%), việc tăng thêm 1% tuyệt đối (thành 2%) thực chất là tăng gấp đôi cơ hội được chọn (100% về mặt tương đối). Ngược lại, nếu hành động đã đạt 90% xác suất, cộng thêm 1% là rất không đáng kể.

Đạo hàm Logarit $\nabla \ln \pi$ nắm bắt chính xác sự thay đổi tỷ lệ phần trăm này. Nó giúp mạng Nơ-ron phải cập nhật mạnh hơn các hành động tốt nhưng đang có xác suất thấp", đồng thời tránh cập nhật quá tay đối với các hành động đã có xác suất cao.

2.2.2. Chứng minh Định lý Policy Gradient

Sử dụng Mẹo Log-derivative, ta biến đổi hàm mục tiêu $J(\theta)$. Bắt đầu bằng việc viết hàm kỳ vọng dưới dạng tổng xác suất:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \sum_{\tau} P(\tau | \theta) R(\tau) = \sum_{\tau} \nabla_{\theta} P(\tau | \theta) R(\tau)$$

Áp dụng Mẹo Logarit cho $P(\tau | \theta)$:

$$\nabla_{\theta} J(\theta) = \sum_{\tau} P(\tau | \theta) \nabla_{\theta} \ln P(\tau | \theta) R(\tau)$$

Chuyển đổi tổng trở lại thành dạng kỳ vọng ($\mathbb{E}$):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \ln P(\tau | \theta) R(\tau)]$$

Xác suất của một quỹ đạo $P(\tau | \theta)$ là tích của xác suất trạng thái ban đầu, động học của môi trường, và xác suất của Policy. Khi lấy Logarit, phép nhân biến thành phép cộng. Khi lấy Gradient theo θ , mọi thành phần thuộc về môi trường (không chứa θ) sẽ bằng 0 và bị triệt tiêu hoàn toàn. Do đó, ta thu được Định lý Policy Gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\sum_{t=0}^T \nabla_{\theta} \ln \pi_{\theta}(a_t | s_t) R(\tau)]$$

Trực quan hóa: Phương trình trên là một kết quả quan trọng của RL. Điều này có nghĩa là: Chúng ta không cần hiểu các định luật vật lý của CartPole. Ta chỉ cần lấy Logarit xác suất hành động do mạng Nơ-ron dự đoán, nhân với tổng phần thưởng $R(\tau)$ đạt được, và tiến hành cập nhật trọng số. Phần thưởng cao sẽ củng cố hành động, phần thưởng thấp sẽ loại bỏ chúng.

2.3. Phân tích chi tiết (Deep Dive): Thuật toán REINFORCE & Giảm Phương sai

Dựa trên Định lý Policy Gradient đã chứng minh, thuật toán REINFORCE ra đời để dịch lý thuyết này thành các bước cập nhật trọng số trong thực tế. Sức mạnh của nó nằm ở việc kết hợp Policy ngẫu nhiên với 3 cơ chế cốt lõi sau.

2.3.1. Tối ưu hóa dốc lên (Gradient Ascent) và "Nghịch lý PyTorch"

Trong Machine Learning truyền thống (như phân loại ảnh), ta dùng Gradient Descent (Hạ Gradient) để tìm điểm thấp nhất của Hàm mất mát (Loss Function) — giống như đi tìm đáy giá trị nhỏ nhất.

Ngược lại, trong RL, mục tiêu $J(\theta)$ của ta là Tổng phần thưởng. Do đó, ta phải dùng Gradient Ascent để leo lên giá trị lớn nhất (nơi có phần thưởng cao nhất):

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Chi tiết hiện thực code: Mặc định, thư viện PyTorch chỉ được thiết kế cho Gradient Descent (tối thiểu hóa). Để "đổi dấu" PyTorch thực hiện Gradient Ascent (tối đa hóa), trong mã nguồn (reinforce.py), chúng ta thêm một dấu âm vào Loss: `policy_loss.append(-log_prob * G_t)`. Tối thiểu hóa một giá trị âm về mặt toán học chính là tối đa hóa giá trị dương của nó.

2.3.2. Tính Nhân quả (Causality) và Phần thưởng tương lai (G_t)

Theo định lý gốc, Gradient được nhân với $R(\tau)$ (tổng phần thưởng của cả ván chơi). Tuy nhiên, điều này bộc lộ một điểm yếu logic: Hành động ở hiện tại không thể làm thay đổi các phần thưởng đã nhận trong quá khứ.

Khắc phục gán tín dụng theo thời gian: Giả sử Agent sống sót được 100 bước. Ở bước $t = 99$, Agent phạm sai lầm chí mạng khiến cột đổ. Nếu dùng tổng $R(\tau)$, hành động tồi tệ ở bước 99 vẫn được nhân với một lượng phần thưởng khổng lồ tích lũy từ bước 1 đến 98. Điều này vô lý và làm nhiễu mạng Nơ-ron.

Để giải quyết, REINFORCE áp dụng Tính nhân quả, thay thế tổng $R(\tau)$ bằng phần thưởng tương lai G_t (Reward-to-go) — chỉ tính tổng phần thưởng từ thời điểm t trở đi:

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

Với sự cải tiến này, nếu một hành động tại thời điểm t dẫn đến kết thúc game ngay lập tức, G_t sẽ rất nhỏ, kéo theo việc giảm mạnh xác suất của hành động sai lầm đó.

2.3.3. Giảm Phương sai bằng Đường cơ sở (Baseline)

Môi trường CartPole-v1 có một đặc thù là phần thưởng luôn dương (+1 cho mỗi bước sống sót). Dẫn đến việc thuật toán về mặt kỹ thuật sẽ luôn "khen" (tăng xác suất) mọi hành động, chỉ khác nhau ở mức độ khen nhiều hay ít.

Trực quan bằng số:

- Quỹ đạo A chơi xuất sắc: $G_t = 500$ (Điểm A+)
- Quỹ đạo B chơi cực tệ: $G_t = 10$ (Điểm F)

Nếu không có Baseline, hệ thống vẫn cộng trọng số cho quỹ đạo B (vì $10 > 0$). Điều này gây ra phương sai (variance) khổng lồ cho gradient, làm quá trình học chậm và thiếu ổn định.

Giải pháp: Ta đưa vào một "Đường cơ sở" $b(s_t)$, ví dụ như mức điểm trung bình là 100. Thuật toán sẽ lấy G_t trừ đi 100:

- Quỹ đạo A: $500 - 100 = +400 \rightarrow$ Khen thưởng (Tăng mạnh xác suất).
- Quỹ đạo B: $10 - 100 = -90 \rightarrow$ Trừng phạt (Kéo giảm xác suất của hành động tồi).

Chứng minh Toán học: Baseline không làm lệch hướng học (Unbiased)

Việc tự ý trừ đi trung bình (như lệnh `returns = returns - returns.mean()` trong code) có làm sai lệch bài toán gốc không? Câu trả lời là KHÔNG. Xét đại lượng dính tới Baseline trong kỳ vọng Gradient:

$$\mathbb{E}_{a_t \sim \pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t | s_t) b(s_t)] = \sum_{a_t} \pi_\theta(a_t | s_t) \frac{\nabla_\theta \pi_\theta(a_t | s_t)}{\pi_\theta(a_t | s_t)} b(s_t)$$

Triệt tiêu π_θ ở tử và mẫu, đưa hằng số Baseline ra ngoài dấu tổng:

$$= b(s_t) \nabla_\theta \sum_{a_t} \pi_\theta(a_t | s_t)$$

Theo định lý xác suất, tổng tất cả các xác suất tại một trạng thái luôn bằng 1 ($\sum \pi = 1$), và đạo hàm của một hằng số thì bằng 0:

$$= b(s_t) \nabla_\theta (1) = b(s_t) \times 0 = 0$$

Tiểu kết: Phép chứng minh thanh lịch này khẳng định rằng: Kỹ thuật Baseline triệt tiêu hoàn toàn phương sai (độ nhiễu), giúp định chuẩn lại đâu là "tốt", đâu là "xấu", nhưng tuyệt đối không làm thay đổi giá trị kỳ vọng (hướng tối ưu chuẩn xác) của Policy Gradient. Đây chính là mảnh ghép cuối cùng giúp thuật toán REINFORCE hội tụ ổn định trên thực tế.

2.4. Kết luận

Thông qua Chương 2, chúng ta đã thiết lập thành công một nền tảng toán học vững chắc để giải quyết bài toán CartPole-v1. Khởi đầu từ việc mô hình hóa môi trường thông qua Quá trình Quyết định Markov (MDP), chúng ta đã chứng minh định lý Policy Gradient nhờ vào Mẹo Đạo hàm Logarit — biến một bài toán tối ưu hóa "không có dạng giải tích tường minh" thành một hàm kỳ vọng có thể đo lường được.

Quan trọng hơn, thông qua quá trình phân tích sâu (Deep Dive) về thuật toán REINFORCE, chúng ta đã kết nối giữa lý thuyết và thực tiễn bằng cách xử lý vấn đề gán tín dụng theo thời gian phần thưởng (sử dụng tính nhân quả G_t) và vấn đề phương sai cao (sử dụng kỹ thuật Baseline Subtraction). Sự gắn kết chặt chẽ giữa Toán học và Logic này chính là nền tảng cốt lõi để chúng ta hiện thực hóa mạng nơ-ron và quy trình huấn luyện mã nguồn trong Chương 3 tiếp theo.